

SIMATSENGINEERING(SAVEETHASCHOOLOFENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Technical Presentation

Course Code:	CSA0305	Course Title:	Data Structures
CO Assessed:	CO1 – Imitation (BL3, BL4)	Assessment:	Assessment Tool 3 – Technical Presentation
Weightage:	35% of CIA	Total Marks:	100
CO1 Statement:	Applies suitable data structures and ADTs for efficient problem solving by analyzing time and space complexity.		
Student Name:	M.VAISHNAV	Reg. No.:	192524251
Section/Batch:		Date:	15/07/2026

Code of Conduct

I M.VAISHNAV (192524251) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: M.VAISHNAV (192524251)

Instructions

- This assessment contains technical presentation. Total: 100 Marks.
- When a student delivers a technical presentation on a data structures topic, evaluation should be based on the following requirements: Concept explanation, Real-world application and Clarity of presentation.
- Demonstrate clear understanding, not just reading slides
- Use of tools: PowerPoint/Google Slides.
- Duration: 7–10 minutes presentation + 3 minutes Q&A.

Section/Topic	Focus	Q Nos	Marks
Data Structure	Real-Time Applications	1	100
GRAND TOTAL:		100 Marks	

Annexure A – Technical Presentation

A web browser supports navigating backward through previously visited pages. Recommend a suitable ADT and prepare a technical presentation explaining how it improves performance efficiency. (100 Marks)

(OR)

A music streaming application maintains a playlist where songs can be added, removed, or reordered dynamically. Recommend a suitable data structure and justify based on flexibility and efficiency. Prepare a technical Presentation. (100 Marks)

(OR)

A metro rail network maps stations and routes to determine reachable stations from a source station. Suggest a suitable data structure for representation and justify your answer. (100 Mark)

(OR)

A calculator application evaluates arithmetic expressions entered by users. Recommend a suitable data structure and justify based on expression processing efficiency. (100 Marks)

(OR)

A cafeteria token system serves customers strictly in the order of arrival. Identify the suitable ADT and justify why it ensures fairness. (100 Marks)

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Concept Understanding	20	Demonstrates deep and accurate understanding of data structure with insights and connections	Shows clear understanding with minor gaps	Basic understanding; some misconceptions present	Limited or incorrect understanding
Content Organization	30	Logical, well-structured, smooth flow; strong introduction and conclusion	Mostly organized with clear flow	Some organization but lacks clarity or transitions	Disorganized, difficult to follow
Technical Depth	20	Includes algorithms, complexity analysis, and advanced insights	Includes algorithm and basic complexity analysis	Limited technical details; missing depth	Minimal or no technical content
PowerPoint Slides	20	Excellent design, clear visuals, enhances understanding	Good slides with minor issues	Basic slides; somewhat cluttered or unclear	Poor slides or no visual support
Communication Skills	10	Clear, confident, engaging delivery; excellent articulation	Clear delivery with minor hesitation	Some hesitation; unclear in parts	Poor communication; difficult to understand

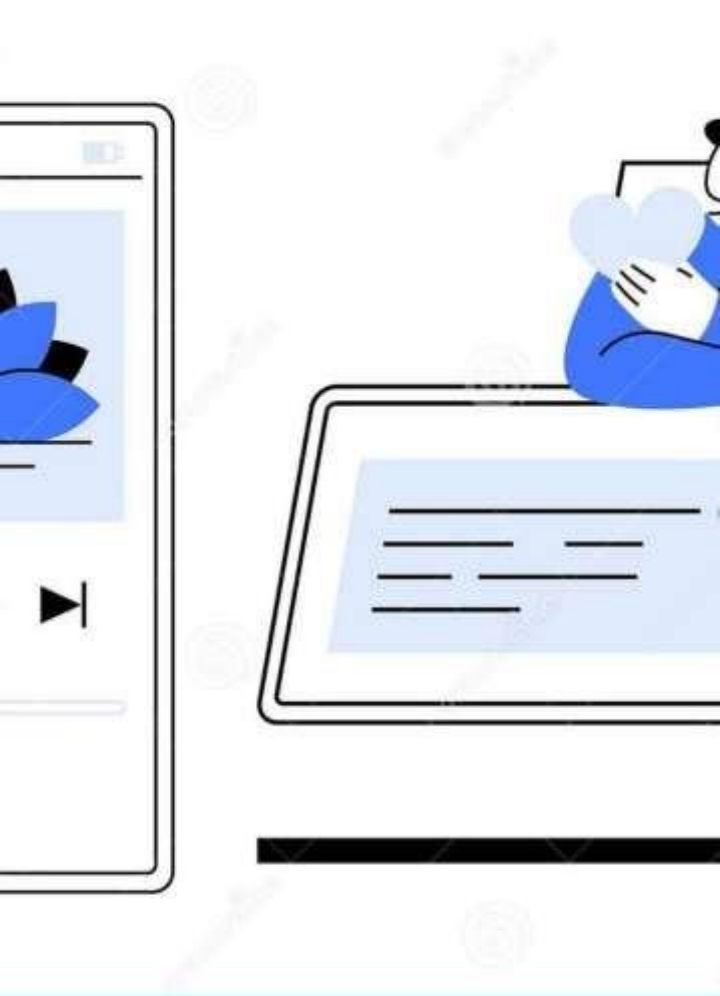
Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____



ΘΚnΘ misPlΘ Klist
MΘ nΘ ⑦ementinMusis
StíeΘ min⑦
6pplisΘ tions Usin⑦
ΘΘtΘ Stíustuíes

EfficientPlaylistOperationsUsingDoublyLinkedLists

Presented by: M.VAISHNAV—Roll No: 192524251—Dept. of Artificial
Intelligence and Data Science, SIMATSE Engineering



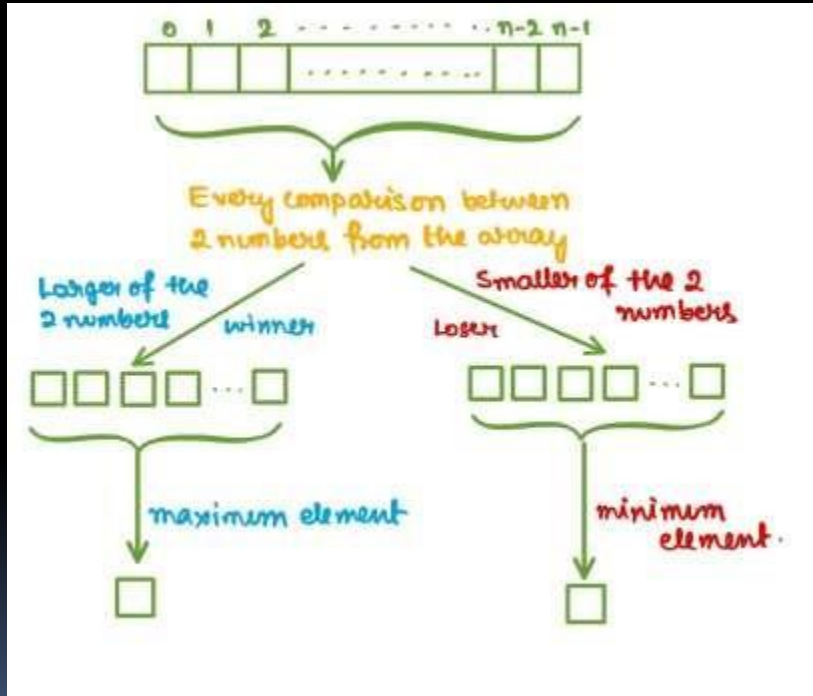
Intío[®]ustion — WhK plΘ Klist peífoím Θ nse m Θ tteís

Streamingservicesmanagelargelibrariesandmillionsofuserplaylists.
Usersexpectinstantresponseswhentheyadd,remove,orreordertracks.
Theplaylistback-endmustprovidelow-latencyoperationswhile
supportingfeatureslikeskip,repeat,andhistory.

Píoblemst Θ tement

- Supportdynamicinsertionoftracksatarbitrarypositions
- Allowdeletionwithminimaloverhead
- Enablereorderingandmoveswithconstant-timeupdates
- ProvidefastNext/Previousnavigationforplayback
- Scaleefficientlyforverylongplaylists

Θ t Θ Stíustuíe C Θ n[®] i[®] Θ tes



6íí Θ K

Pros: $O(1)$ random access, compact memory layout. Cons: insertion/deletion requires shifting elements— $O(n)$ worst-case; expensive for frequent edits.

Sin 7IK Linke[®] List

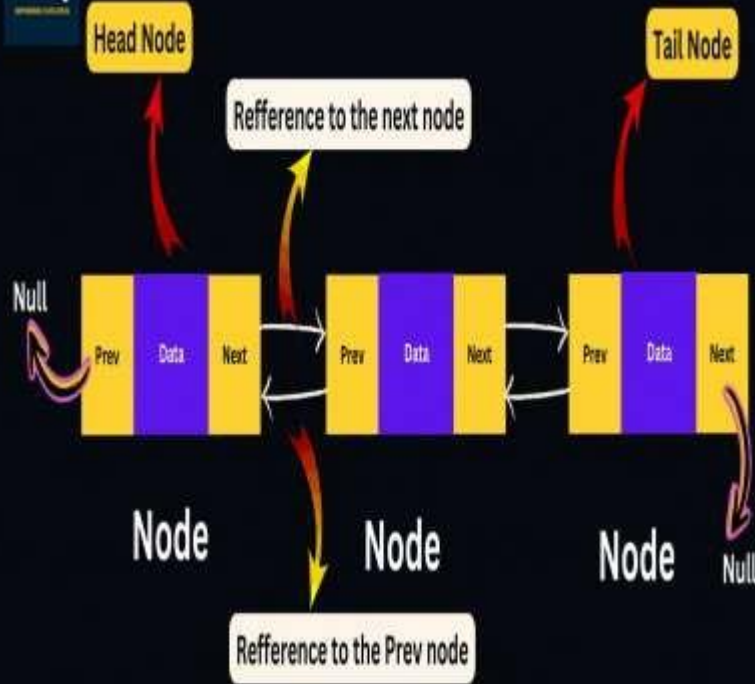
Pros: $O(1)$ insertion at known node, low-cost node allocation. Cons: no backward traversal, deletion requires pointer to previous node or $O(n)$ search.

ΘoubIK Linke[®] List

Pros: $O(1)$ insertion & deletion given node reference, forward and backward traversal, ideal for playNext/Previous and efficient reordering. Slightly higher memory (extra pointer) but better operational fit for playlists.



Doubly Linked List



Resommen[®]e[®]Stíustuíe— DoubKLinke[®]List

Each playlist node stores: previous pointer, song metadata (id, title, artist, duration, URI), and next pointer. The list supports head/tail references and optional hashing (indexmap) for $O(1)$ lookup of nodes by song id.

No[®]e
sontents

Prev pointer ·
Songdata · Next
pointer

6uxili Θ íK
stíustuíes

HashMap: songId
→ node (optional)
for direct access

Pl Θ Klist
fe Θ tuíes

Skip, repeat,
move-to-front,
remove, insert-
after, insert-
before

Why Double-Linked Lists Win

Bi-directional
navigation

Native support for Next
and Previous playback
without extra state.



Constant-time
updates

Insert, remove, and move
require only pointer
updates when node
reference known.



Integrates with
Kubernetes

Works well with diffs,
CRDTs, and server-side
persistence for large-scale
apps.



Conclusion & Recommendation

Doubly linked lists provide the best balance of efficiency, flexibility, and straightforward implementation for dynamic playlist management in music streaming apps. When combined with an optional songId $\rightarrow$ node map and a compact sync log, they deliver $O(1)$ runtime edits and robust cross-device behavior.

Final recommendation: Implement an in-memory doubly linked list for runtime playlist operations, augment with an index map for direct access, and integrate a lightweight event log for synchronization and durability.

Thank you



Dynamic Playlist Management in Music Streaming Applications Using Data Structures

Efficient Playlist Operations Using Doubly Linked Lists

Presented by: M. VAISHNAV - Roll No. 192524251 - Dept. of Artificial Intelligence and Data Science, SIMATS Engineering